lezione-30-04-24

September 22, 2024

Laboratorio di WebScraping

Anno Accademico 2023-2024

Docente: Laura Ricci

Lezione 21

Ottimizzare l'uso di memoria in PANDAS

30 Aprile 2024

0.1 Ripasso: Indexing and Slicing Data Frames

- per selezionare dati da un DataFrame:
  - [ ]
  - **.loc**[ ]
  - **.iloc**[ ]
  - **Boolean indexing**
- riassumiamoli considerando il seguente **DataFrame**

[1]:
```
import pandas as pd
df = pd.DataFrame({"Name": ["Laura", "Giovanni", "Anna"],
                   "Language": ["Python", "Python", "R"],
                   "Courses": [5, 4, 7]})
df
```

[1]:
```
       Name Language  Courses
0     Laura   Python        5
1  Giovanni   Python        4
2      Anna        R        7
```

0.2 Indexing Columns: [ ]

[14]:
```
df
```

[14]:
```
       Name Language  Courses
0     Laura   Python        5
1  Giovanni   Python        4
2      Anna        R        7
```

[2]:
```
df['Name']  # restituisce una Serie
```

[2]:
```
0       Laura
1    Giovanni
2        Anna
Name: Name, dtype: object
```

[3]:
```
df[['Name']]  # restituisce un DataFrame!
```

[3]:
```
        Name
0     Laura
1  Giovanni
2      Anna
```

0.3 Indexing Columns: [ ]

[15]:
```
df
```

[15]:
```
        Name Language  Courses
0     Laura   Python        5
1  Giovanni   Python        4
2      Anna        R        7
```

[4]:
```
df[['Name', 'Language']]
```

[4]:
```
        Name Language
0     Laura   Python
1  Giovanni   Python
2      Anna        R
```

0.4 Indexing Rows [ ]

[17]:
```
df
```

[17]:
```
        Name Language  Courses
0     Laura   Python        5
1  Giovanni   Python        4
2      Anna        R        7
```

[5]:
```
df[0] # non funziona!
```

```
---------------------------------------------------------------------------
KeyError                                  Traceback (most recent call last)
File c:
 ↪\users\ricci\appdata\local\programs\python\python38\lib\site-packages\pandas\core\indexes\
 ↪py:3802, in Index.get_loc(self, key, method, tolerance)
   3801 try:
```

```
-> 3802     return self._engine.get_loc(casted_key)
   3803 except KeyError as err:

File c:
 ↪\users\ricci\appdata\local\programs\python\python38\lib\site-packages\pandas\_libs\index.
 ↪pyx:138, in pandas._libs.index.IndexEngine.get_loc()

File c:
 ↪\users\ricci\appdata\local\programs\python\python38\lib\site-packages\pandas\_libs\index.
 ↪pyx:165, in pandas._libs.index.IndexEngine.get_loc()

File pandas\_libs\hashtable_class_helper.pxi:5745, in pandas._libs.hashtable.
 ↪PyObjectHashTable.get_item()

File pandas\_libs\hashtable_class_helper.pxi:5753, in pandas._libs.hashtable.
 ↪PyObjectHashTable.get_item()

KeyError: 0

The above exception was the direct cause of the following exception:

KeyError                                  Traceback (most recent call last)
Cell In[5], line 1
----> 1 df[0] # non funziona!

File c:
 ↪\users\ricci\appdata\local\programs\python\python38\lib\site-packages\pandas\core\frame.
 ↪py:3807, in DataFrame.__getitem__(self, key)
   3805 if self.columns.nlevels > 1:
   3806     return self._getitem_multilevel(key)
-> 3807 indexer = self.columns.get_loc(key)
   3808 if is_integer(indexer):
   3809     indexer = [indexer]

File c:
 ↪\users\ricci\appdata\local\programs\python\python38\lib\site-packages\pandas\core\indexes\
 ↪py:3804, in Index.get_loc(self, key, method, tolerance)
   3802     return self._engine.get_loc(casted_key)
   3803 except KeyError as err:
-> 3804     raise KeyError(key) from err
   3805 except TypeError:
   3806     # If we have a listlike key, _check_indexing_error will raise
   3807     #  InvalidIndexError. Otherwise we fall through and re-raise
   3808     #  the TypeError.
   3809     self._check_indexing_error(key)

KeyError: 0
```

0.5 Indexing Rows: [ ]

[18]:
```
df
```

[18]:
```
       Name Language  Courses
0     Laura   Python        5
1  Giovanni   Python        4
2      Anna        R        7
```

[6]:
```
df[0:1] # funziona!
```

[6]:
```
    Name Language  Courses
0  Laura   Python        5
```

[7]:
```
df[1:] # funziana!
```

[7]:
```
       Name Language  Courses
1  Giovanni   Python        4
2      Anna        R        7
```

0.6 Indexing: .loc e .iloc

- sono i metodi preferibili!

[19]:
```
df
```

[19]:
```
       Name Language  Courses
0     Laura   Python        5
1  Giovanni   Python        4
2      Anna        R        7
```

[8]:
```
df.iloc[0]  # returns a series
```

[8]:
```
Name          Laura
Language     Python
Courses           5
Name: 0, dtype: object
```

[ ]:
```
df.iloc[0:2]  # slicing returns a dataframe
```

0.7 Indexing: .iloc

[20]:
```
df
```

[20]:
```
       Name Language  Courses
0     Laura   Python        5
1  Giovanni   Python        4
```

```
2      Anna        R        7
```

[9]:
```
df.iloc[2, 1]  # returns the indexed object
```

[9]:
```
'R'
```

[10]:
```
df.iloc[[0, 1], [1, 2]]  # returns a dataframe
```

[10]:
```
  Language  Courses
0   Python        5
1   Python        4
```

0.8 Indexing: .loc

[21]:
```
df
```

[21]:
```
        Name Language  Courses
0     Laura   Python        5
1  Giovanni   Python        4
2      Anna        R        7
```

[11]:
```
df.loc[:, 'Name']
```

[11]:
```
0       Laura
1    Giovanni
2        Anna
Name: Name, dtype: object
```

[12]:
```
df.loc[:, 'Name':'Language']
```

[12]:
```
        Name Language
0     Laura   Python
1  Giovanni   Python
2      Anna        R
```

0.9 Indexing: .loc

[23]:
```
df
```

[23]:
```
        Name Language  Courses
0     Laura   Python        5
1  Giovanni   Python        4
2      Anna        R        7
```

[13]:
```
df.loc[[0, 2], ['Language']]
```

[13]:
```
  Language
0   Python
2        R
```

0.10 Boolean Indexing

[24]:
```
df
```

[24]:
```
       Name Language  Courses
0     Laura   Python        5
1  Giovanni   Python        4
2      Anna        R        7
```

[25]:
```
df[df['Courses'] > 5]
```

[25]:
```
   Name Language  Courses
2  Anna        R        7
```

[27]:
```
df[df['Name'] == "Giovanni"]
```

[27]:
```
       Name Language  Courses
1  Giovanni   Python        4
```

0.11 Ripasso: DataFrame Summaries

[28]:
```
import pandas as pd
df_cycling = pd.read_csv("DataSets/cycling_data.csv", parse_dates=True)
df_cycling
```

[28]:
```
                      Date            Name  Type   Time  Distance  \
0   10 Sep 2019, 00:13:04  Afternoon Ride  Ride   2084     12.62
1   10 Sep 2019, 13:52:18    Morning Ride  Ride   2531     13.03
2   11 Sep 2019, 00:23:50  Afternoon Ride  Ride   1863     12.52
3   11 Sep 2019, 14:06:19    Morning Ride  Ride   2192     12.84
4   12 Sep 2019, 00:28:05  Afternoon Ride  Ride   1891     12.48
5   16 Sep 2019, 13:57:48    Morning Ride  Ride   2272     12.45
6   17 Sep 2019, 00:15:47  Afternoon Ride  Ride   1973     12.45
7   17 Sep 2019, 13:43:34    Morning Ride  Ride   2285     12.60
8   18 Sep 2019, 13:49:53    Morning Ride  Ride   2903     14.57
9   18 Sep 2019, 00:15:52  Afternoon Ride  Ride   2101     12.48
10  19 Sep 2019, 00:30:01  Afternoon Ride  Ride  48062     12.48
11  19 Sep 2019, 13:52:09    Morning Ride  Ride   2090     12.59
12  20 Sep 2019, 01:02:05  Afternoon Ride  Ride   2961     12.81
13  23 Sep 2019, 13:50:41    Morning Ride  Ride   2462     12.68
14  24 Sep 2019, 00:35:42  Afternoon Ride  Ride   2076     12.47
15  24 Sep 2019, 13:41:24    Morning Ride  Ride   2321     12.68
```

16	25 Sep 2019, 00:07:21	Afternoon Ride	Ride	1775	12.10
17	25 Sep 2019, 13:35:41	Morning Ride	Ride	2124	12.65
18	26 Sep 2019, 00:13:33	Afternoon Ride	Ride	1860	12.52
19	26 Sep 2019, 13:42:43	Morning Ride	Ride	2350	12.91
20	27 Sep 2019, 01:00:18	Afternoon Ride	Ride	1712	12.47
21	30 Sep 2019, 13:53:52	Morning Ride	Ride	2118	12.71
22	1 Oct 2019, 00:15:07	Afternoon Ride	Ride	1732	NaN
23	1 Oct 2019, 13:45:55	Morning Ride	Ride	2222	12.82
24	2 Oct 2019, 00:13:09	Afternoon Ride	Ride	1756	NaN
25	2 Oct 2019, 13:46:06	Morning Ride	Ride	2134	13.06
26	3 Oct 2019, 00:45:22	Afternoon Ride	Ride	1724	12.52
27	3 Oct 2019, 13:47:36	Morning Ride	Ride	2182	12.68
28	4 Oct 2019, 01:08:08	Afternoon Ride	Ride	1870	12.63
29	9 Oct 2019, 13:55:40	Morning Ride	Ride	2149	12.70
30	10 Oct 2019, 00:10:31	Afternoon Ride	Ride	1841	12.59
31	10 Oct 2019, 13:47:14	Morning Ride	Ride	2463	12.79
32	11 Oct 2019, 00:16:57	Afternoon Ride	Ride	1843	11.79

	Comments
0	Rain
1	rain
2	Wet road but nice weather
3	Stopped for photo of sunrise
4	Tired by the end of the week
5	Rested after the weekend!
6	Legs feeling strong!
7	Raining
8	Raining today
9	Pumped up tires
10	Feeling good
11	Getting colder which is nice
12	Feeling good
13	Rested after the weekend!
14	Oiled chain, bike feels smooth
15	Bike feeling much smoother
16	Feeling really tired
17	Stopped for photo of sunrise
18	raining
19	Detour around trucks at Jericho
20	Tired by the end of the week
21	Rested after the weekend!
22	Legs feeling strong!
23	Beautiful morning! Feeling fit
24	A little tired today but good weather
25	Bit tired today but good weather
26	Feeling good
27	Wet road

```
28           Very tired, riding into the wind
29              Really cold! But feeling good
30        Feeling good after a holiday break!
31               Stopped for photo of sunrise
32  Bike feeling tight, needs an oil and pump
```

0.12 Ripasso: DataFrame Summaries

[29]:
```
df_cycling.set_index("Date")
```

[29]:

Date	Name	Type	Time	Distance	Comments
10 Sep 2019, 00:13:04	Afternoon Ride	Ride	2084	12.62	
10 Sep 2019, 13:52:18	Morning Ride	Ride	2531	13.03	
11 Sep 2019, 00:23:50	Afternoon Ride	Ride	1863	12.52	
11 Sep 2019, 14:06:19	Morning Ride	Ride	2192	12.84	
12 Sep 2019, 00:28:05	Afternoon Ride	Ride	1891	12.48	
16 Sep 2019, 13:57:48	Morning Ride	Ride	2272	12.45	
17 Sep 2019, 00:15:47	Afternoon Ride	Ride	1973	12.45	
17 Sep 2019, 13:43:34	Morning Ride	Ride	2285	12.60	
18 Sep 2019, 13:49:53	Morning Ride	Ride	2903	14.57	
18 Sep 2019, 00:15:52	Afternoon Ride	Ride	2101	12.48	
19 Sep 2019, 00:30:01	Afternoon Ride	Ride	48062	12.48	
19 Sep 2019, 13:52:09	Morning Ride	Ride	2090	12.59	
20 Sep 2019, 01:02:05	Afternoon Ride	Ride	2961	12.81	
23 Sep 2019, 13:50:41	Morning Ride	Ride	2462	12.68	
24 Sep 2019, 00:35:42	Afternoon Ride	Ride	2076	12.47	
24 Sep 2019, 13:41:24	Morning Ride	Ride	2321	12.68	
25 Sep 2019, 00:07:21	Afternoon Ride	Ride	1775	12.10	
25 Sep 2019, 13:35:41	Morning Ride	Ride	2124	12.65	
26 Sep 2019, 00:13:33	Afternoon Ride	Ride	1860	12.52	
26 Sep 2019, 13:42:43	Morning Ride	Ride	2350	12.91	
27 Sep 2019, 01:00:18	Afternoon Ride	Ride	1712	12.47	
30 Sep 2019, 13:53:52	Morning Ride	Ride	2118	12.71	
1 Oct 2019, 00:15:07	Afternoon Ride	Ride	1732	NaN	
1 Oct 2019, 13:45:55	Morning Ride	Ride	2222	12.82	
2 Oct 2019, 00:13:09	Afternoon Ride	Ride	1756	NaN	
2 Oct 2019, 13:46:06	Morning Ride	Ride	2134	13.06	
3 Oct 2019, 00:45:22	Afternoon Ride	Ride	1724	12.52	
3 Oct 2019, 13:47:36	Morning Ride	Ride	2182	12.68	
4 Oct 2019, 01:08:08	Afternoon Ride	Ride	1870	12.63	
9 Oct 2019, 13:55:40	Morning Ride	Ride	2149	12.70	
10 Oct 2019, 00:10:31	Afternoon Ride	Ride	1841	12.59	
10 Oct 2019, 13:47:14	Morning Ride	Ride	2463	12.79	
11 Oct 2019, 00:16:57	Afternoon Ride	Ride	1843	11.79	

```
Date
10 Sep 2019, 00:13:04                                    Rain
10 Sep 2019, 13:52:18                                    rain
11 Sep 2019, 00:23:50                  Wet road but nice weather
11 Sep 2019, 14:06:19                Stopped for photo of sunrise
12 Sep 2019, 00:28:05                Tired by the end of the week
16 Sep 2019, 13:57:48                   Rested after the weekend!
17 Sep 2019, 00:15:47                        Legs feeling strong!
17 Sep 2019, 13:43:34                                    Raining
18 Sep 2019, 13:49:53                              Raining today
18 Sep 2019, 00:15:52                            Pumped up tires
19 Sep 2019, 00:30:01                               Feeling good
19 Sep 2019, 13:52:09                Getting colder which is nice
20 Sep 2019, 01:02:05                               Feeling good
23 Sep 2019, 13:50:41                   Rested after the weekend!
24 Sep 2019, 00:35:42              Oiled chain, bike feels smooth
24 Sep 2019, 13:41:24                  Bike feeling much smoother
25 Sep 2019, 00:07:21                        Feeling really tired
25 Sep 2019, 13:35:41                Stopped for photo of sunrise
26 Sep 2019, 00:13:33                                    raining
26 Sep 2019, 13:42:43             Detour around trucks at Jericho
27 Sep 2019, 01:00:18                Tired by the end of the week
30 Sep 2019, 13:53:52                   Rested after the weekend!
1 Oct 2019, 00:15:07                         Legs feeling strong!
1 Oct 2019, 13:45:55               Beautiful morning! Feeling fit
2 Oct 2019, 00:13:09       A little tired today but good weather
2 Oct 2019, 13:46:06            Bit tired today but good weather
3 Oct 2019, 00:45:22                               Feeling good
3 Oct 2019, 13:47:36                                   Wet road
4 Oct 2019, 01:08:08            Very tired, riding into the wind
9 Oct 2019, 13:55:40               Really cold! But feeling good
10 Oct 2019, 00:10:31        Feeling good after a holiday break!
10 Oct 2019, 13:47:14              Stopped for photo of sunrise
11 Oct 2019, 00:16:57  Bike feeling tight, needs an oil and pump
```

0.13 DataFrame Summaries: shape

- tre funzioni molto utili per reperire informazioni sui data frame
  - **shape**
  - **info()**
  - **describe()**

[30]:
```
df_cycling.shape
```

[30]:
```
(33, 6)
```

- restituisce una tupla con le dimensioni del **DataFrame**
- è una proprietà dell'oggetto **DataFrame**

0.14 DataFrame Summaries: info

[31]:
```
df_cycling.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 33 entries, 0 to 32
Data columns (total 6 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   Date      33 non-null     object
 1   Name      33 non-null     object
 2   Type      33 non-null     object
 3   Time      33 non-null     int64
 4   Distance  31 non-null     float64
 5   Comments  33 non-null     object
dtypes: float64(1), int64(1), object(4)
memory usage: 1.7+ KB
```

- il metodo **df.info()** fornisce informazioni relative a
  - ogni colonna del **DataFrame**
  - l"**uso della memoria**
  - ed altro...

0.15 DataFrame Summaries: describe

[32]:
```
df_cycling.describe(include='all')
```

[32]:

	Date	Name	Type	Time	Distance	Comments
count	33	33	33	33.000000	31.000000	33
unique	33	2	1	NaN	NaN	25
top	10 Sep 2019, 00:13:04	Afternoon Ride	Ride	NaN	NaN	Stopped for photo of sunrise
freq	1	17	33	NaN	NaN	3
mean	NaN	NaN	NaN	3512.787879	12.667419	NaN
std	NaN	NaN	NaN	8003.309233	0.428618	NaN
min	NaN	NaN	NaN	1712.000000	11.790000	NaN
25%	NaN	NaN	NaN	1863.000000	12.480000	NaN
50%	NaN	NaN	NaN	2118.000000	12.620000	NaN
75%	NaN	NaN	NaN	2285.000000	12.750000	NaN
max	NaN	NaN	NaN	48062.000000	14.570000	NaN

```
25%                             NaN
50%                             NaN
75%                             NaN
max                             NaN
```

- calcola un insieme di statistiche per tutte le **colonne che contengono valori numerici** nel **DataFrame** (o in una **Series**)
  - numero di elementi, deviazione standard. minimo, quartili, e massimo
  - le informazioni non nemeriche vengono ignorate, se non esplicitamente richiesto di riportarle
- restituisce un **DataFrame**
  - contiene le **informazioni statistiche** calcolate

0.16 DataFrame: gestire i valori nulli

[33]:
```
import numpy as np
import pandas as pd
df = pd.DataFrame([[np.nan, 2, np.nan, 0],
                   [3, 4, np.nan, 1],
                   [np.nan, np.nan, np.nan, 5],
                   [np.nan, 3, np.nan, 4]],
                   columns=list('ABCD'))
df
```

[33]:

	A	B	C	D
0	NaN	2.0	NaN	0
1	3.0	4.0	NaN	1
2	NaN	NaN	NaN	5
3	NaN	3.0	NaN	4

0.17 DataFrame: Individuazione dei valori nulli

[34]:
```
df.isnull()
```

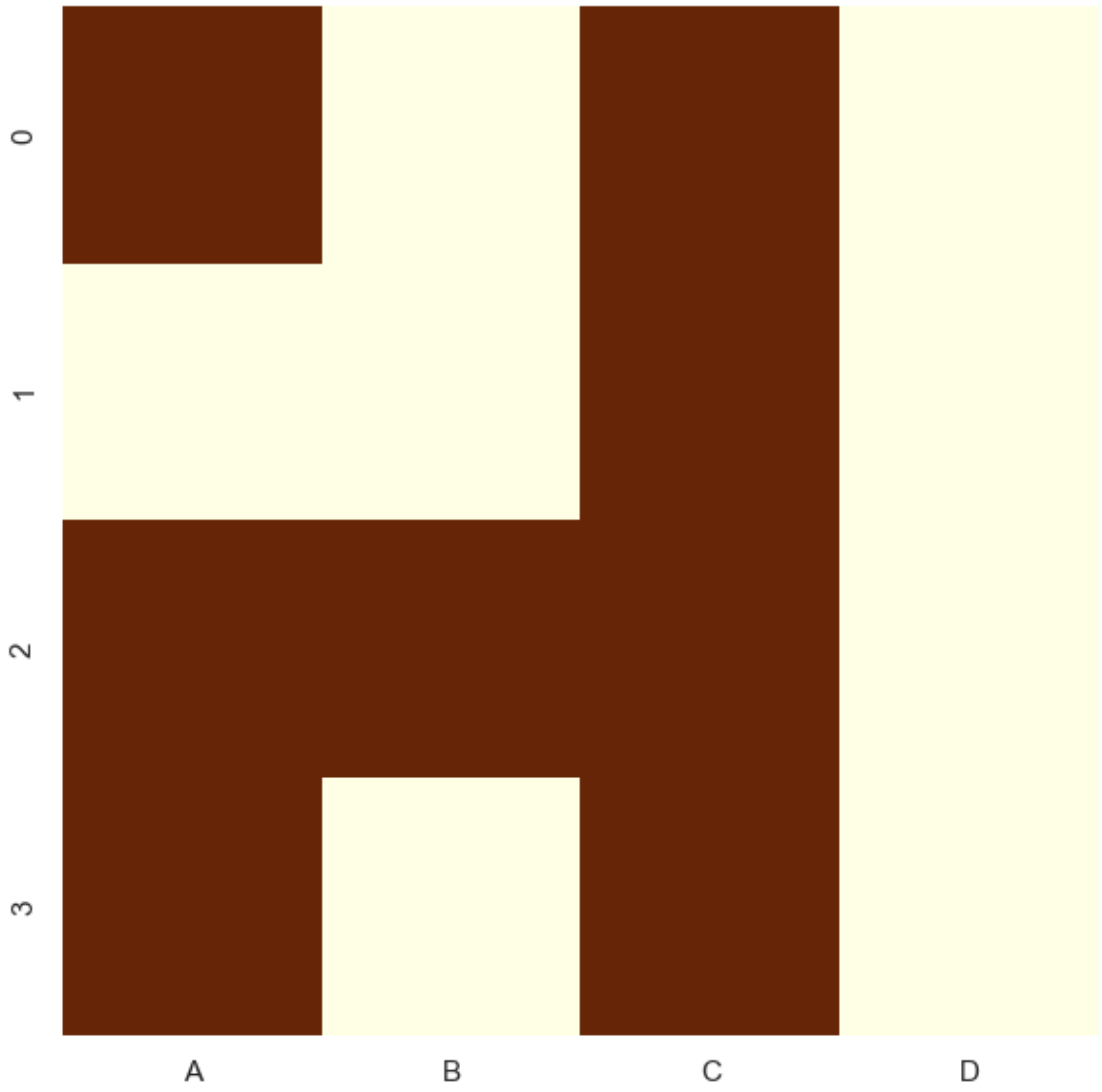
[34]:

	A	B	C	D
0	True	False	True	False
1	False	False	True	False
2	True	True	True	False
3	True	False	True	False

- restituisce un **DataFrame** di valori **Booleani**
  - per ogni posizione del **DataFrame** originario, un valore booleano che indica se il valore corrispondente è **NaN**

0.18 DataFrame: visualizzare i valori nulli con seaborn

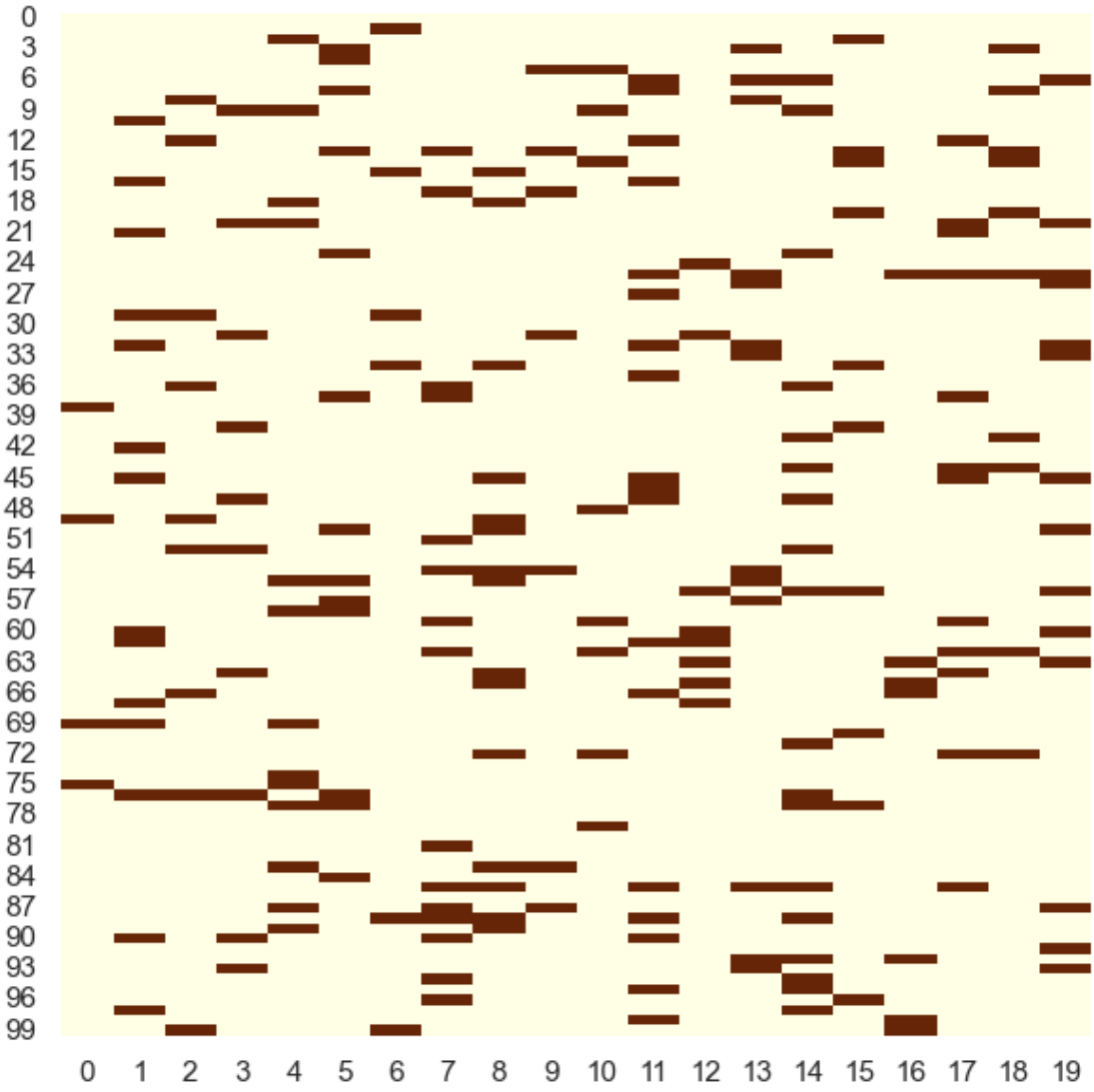
[35]:
```
import seaborn as sns
sns.set(rc={'figure.figsize':(7, 7)})
sns.heatmap(df.isnull(), cmap='YlOrBr', cbar=False);
```



0.19 DataFrame: visualizzare i valori nulli con seaborn

[36]:
```
# Generate a larger synthetic dataset for demonstration
np.random.seed(2020)
npx = np.zeros((100,20))
mask = np.random.choice([True, False], npx.shape, p=[.1, .9])
npx[mask] = np.nan
```

```
sns.heatmap(pd.DataFrame(npx).isnull(), cmap='YlOrBr', cbar=False);
```



0.20 DataFrame: identificazione dei valori nulli

[37]:
```
df_cycling.isnull().sum()
```

[37]:
```
Date        0
Name        0
Type        0
Time        0
Distance    2
Comments    0
dtype: int64
```

- per **DataFrame** di grosse dimensioni, il metodo precedente è applicabile solo a porzioni del DatFrame
- in alternativa, è possibile ottenere informazioni sintetiche (anche co la funzione **info**)

0.21 Gestione dei valori nulli: rimozione

[38]:
```
df.dropna(axis=1)
```

[38]:
```
   D
0  0
1  1
2  5
3  4
```

0.22 Gestione dei valori nulli: valori costanti

[39]:
```
df.fillna(0)
```

[39]:
```
     A    B    C  D
0  0.0  2.0  0.0  0
1  3.0  4.0  0.0  1
2  0.0  0.0  0.0  5
3  0.0  3.0  0.0  4
```

- assegnare al valore nullo una costante

0.23 Gestione dei valori nulli: valori stimati

[40]:
```
df.fillna(df.mean())  # fill with the mean
```

[40]:
```
     A    B   C  D
0  3.0  2.0 NaN  0
1  3.0  4.0 NaN  1
2  3.0  3.0 NaN  5
3  3.0  3.0 NaN  4
```

0.24 Gestione dei valori nulli: forwardfill

[41]:
```
df
```

[41]:
```
     A    B   C  D
0  NaN  2.0 NaN  0
1  3.0  4.0 NaN  1
2  NaN  NaN NaN  5
3  NaN  3.0 NaN  4
```

[42]:
```
df.ffill()
```

[42]:
```
     A    B   C  D
0  NaN  2.0 NaN  0
1  3.0  4.0 NaN  1
2  3.0  4.0 NaN  5
3  3.0  3.0 NaN  4
```

- propaga **in avanti** l'ultimo valore non nullo incontrato

0.25 Gestione dei valori nulli: backwardfill

[43]:
```
df
```

[43]:
```
     A    B   C  D
0  NaN  2.0 NaN  0
1  3.0  4.0 NaN  1
2  NaN  NaN NaN  5
3  NaN  3.0 NaN  4
```

[44]:
```
df.bfill()
```

[44]:
```
     A    B   C  D
0  3.0  2.0 NaN  0
1  3.0  4.0 NaN  1
2  NaN  3.0 NaN  5
3  NaN  3.0 NaN  4
```

- propaga **in indietro** l'ultimo valore non nullo incontrato

0.26 Gestione dei valori nulli: interpolazione

[45]:
```
df
```

[45]:
```
     A    B   C  D
0  NaN  2.0 NaN  0
1  3.0  4.0 NaN  1
2  NaN  NaN NaN  5
3  NaN  3.0 NaN  4
```

[46]:
```
# Interpolate null values linearly
df_interpolated = df.interpolate()
df_interpolated
```

[46]:
```
     A    B   C  D
0  NaN  2.0 NaN  0
1  3.0  4.0 NaN  1
2  3.0  3.5 NaN  5
3  3.0  3.0 NaN  4
```

- interpolazione in base al valore dei vicini
- tecniche più sofisticate utilizzano **machine learning**, tramite **scikit-learn**

0.27 Applying custom functions

[ ]:
```
import pandas as pd
df = pd.read_csv("DataSets/cycling_data.csv", parse_dates=True)
df
```

0.28 Applying custom functions

[ ]:
```
def seconds_to_hours(x):
    return x / 3600

df[['Time']].apply(seconds_to_hours)
```

0.29 Applying custom functions

[ ]:
```
df[['Time']].apply(lambda x: x / 3600)
```

0.30 Applying custom functions

[ ]:
```
def convert_seconds(x, to="hours"):
    if to == "hours":
        return x / 3600
    elif to == "minutes":
        return x / 60

df[['Time']].apply(convert_seconds, to="minutes")
```

0.31 PANDAS: memory model

- il modello di memoria di **Pandas** ha origine da quello di **numpy**, ma verrà modificato a partire dalla versione **3.0** e dalle versione **2.0** sarà possibile scegliere il nuovo modello
- numpy
  - quando si crea un nuovo array a partire da uno esistente, il nuovo è **una vista** sull'array originario
  - ogni modifica effettuta sull'array originario si ripercuote sulla vista e viceversa
- in **PANDAS**, il comportamento è simile
  - infatti **Series** e **DataFrame** sono **"backed"** da un array numpy

[47]:
```
import pandas as pd
print(pd.__version__)
```

```
1.5.3
```

0.32 PANDAS: memory model

- senza la opzione **CopyOnWrite** il comportamento di **Pandas** è molto difficile da prevedere
  - alcune operazioni **Pandas** restituiscono una **copia** della struttura dati, altre una **view**
- se un **oggetto A** è la copia di un **oggetto B** allora ognuno fa riferimento ad una zona di memoria diversa
  - quando modifico la copia non modifico il dato originario
- se un **oggetto A** è una **view** dell'oggetto **B**, allora condividono la stessa zono di memoria
  - ogni modifica a **A**/**B** ha effetti sull'altro array
- molto difficile prevedere quando **Pandas** effettua una copia e quando invece **una view** nella versione attuale
- ogni soluzione presenta degli svantaggi:
  - **view**: possibilità di effetti laterali non desiderati nel codice
  - **copia**: **bassa efficienza** se si lavora con **DataSet** grandi, è possibile che non si voglia

0.33 DataFrame: copy or view?

[48]:
```
import pandas as pd

# This is the default option so I don't strictly
# need this command, but I'll add it to be explicit
pd.set_option("mode.copy_on_write", False)

df = pd.DataFrame({"a": [10, 20, 30, 40], "b": [11, 12, 13, 14]})
df
```

[48]:

	a	b
0	10	11
1	20	12
2	30	13
3	40	14

[49]:
```
my_slice = df.iloc[1:3,]
my_slice
```

[49]:

	a	b
1	20	12
2	30	13

0.34 DataFrame: copy or view?

[50]:
```
df.iloc[1, 1] = -1
df
```

[50]:

	a	b
0	10	11
1	20	-1

```
2  30  13
3  40  14
```

[51]:
```
my_slice
```

[51]:
```
    a   b
1  20  -1
2  30  13
```

- l'esecuzione del programma ha prodotto una **view** sul DataFrame originario
- i due **DataFrame** condividono lo stesso spazio di memoria

0.35 DataFrame: copy or view?

[52]:
```
# Same DataFrame and subset:
df = pd.DataFrame({"a": [10, 20, 30, 40], "b": [11, 12, 13, 14]})
my_slice = df.iloc[1:3,]
my_slice
```

[52]:
```
    a   b
1  20  12
2  30  13
```

[53]:
```
# But now we set the value to 3.14 instead of -1.
df.iloc[1, 1] = 3.14
df
```

[53]:
```
    a      b
0  10  11.00
1  20   3.14
2  30  13.00
3  40  14.00
```

[54]:
```
my_slice
```

[54]:
```
    a   b
1  20  12
2  30  13
```

- diversi comportamenti a seconda del tipo dell'elemento assegnato

0.36 DataFrame: copia esplicita

[55]:
```
df = pd.DataFrame({"a": [10, 20, 30, 40], "b": [11, 12, 13, 14]})
my_slice = df.iloc[1:3,].copy()
my_slice
```

[55]:
```
    a   b
1  20  12
2  30  13
```

[56]:
```
df.iloc[1, 1] = -1
df
```

[56]:
```
    a   b
0  10  11
1  20  -1
2  30  13
3  40  14
```

[57]:
```
my_slice
```

[57]:
```
    a   b
1  20  12
2  30  13
```

- posso **copiare esplicitamente** il DataFrame

0.37 DataFrame: Copy on Write (CoW)

- le **views** sono un meccanismo che consentono di migliorare l'uso della memoria
- i **DataFrame** sono oggetti di dimensione elevata, quindi fare delle copie costa molto, in termini di **memory consumption**
- le regole di **Panda** non sono chiare, ed il comportamento del programma può risultare non predicibile
- la soluzione che verrà adottata a partire da **PANDAS 3.0** sarà la **Copy on Write**
  - cerca di ottenere allo stesso tempo un buon **memory usage** e una **semantica chiara**
- dal punto di vista dell'utente
  - Pandas si comporta cone tutte le operazioni su un **DataFrame** restituissero una **copia**
- dal punto di vista della implementazione
  - **Pandas** non genera **effettivamente** una copia
  - si crea una **view** e solo se viene apportata una modifica ad uno dei due **DataFrame**, viene allora effettata una **copia**

0.38 DataFrame: Copy on Write (CoW)

[61]:
```
import pandas as pd
# settare l'opzione copy_on_write
pd.set_option("mode.copy_on_write", True)
df = pd.DataFrame({"a": [10, 20, 30, 40], "b": [11, 12, 13, 14]})
df
```

[61]:
```
    a   b
0  10  11
1  20  12
```

```
2  30  13
3  40  14
```

[62]:
```
my_slice = df.iloc[1:3,]
my_slice
```

[62]:
```
    a   b
1  20  12
2  30  13
```

0.39 DataFrame: Copy on Write (CoW)

[63]:
```
df.iloc[1, 1] = -1
df
```

[63]:
```
    a   b
0  10  11
1  20  -1
2  30  13
3  40  14
```

[64]:
```
my_slice
```

[64]:
```
    a   b
1  20  12
2  30  13
```

0.40 DataFrame: Copy on Write (CoW)

[65]:
```
# But now we set the value to 3.14 instead of -1.
df.iloc[1, 1] = 3.14
df
```

[65]:
```
    a      b
0  10  11.00
1  20   3.14
2  30  13.00
3  40  14.00
```

[66]:
```
my_slice
```

[66]:
```
    a   b
1  20  12
2  30  13
```

0.41 Ridurre l'uso della RAM in Pandas: scegliere i tipi

- primo, semplice, accorgimento: utilizzare tipi che richiedano poca memoria
- valori numerici
  - **Pandas** per default memorizza i valori interi come **int64** e i valori decimali come **float64**
- **Downcasting**
  - memorizzare **int64** come **int16** o **int8**
  - memorizzare **float64** come **float16** o **float8**
- utilizzando la funzione **astype**

0.42 Profilare la memoria in Pandas

[75]:
```
import pandas as pd
athletes = pd.read_csv('DataSets/athlete_events.csv')
athletes
```

[75]:
```
             ID                      Name Sex   Age  Height  Weight  \
0             1                 A Dijiang   M  24.0   180.0    80.0
1             2                  A Lamusi   M  23.0   170.0    60.0
2             3       Gunnar Nielsen Aaby   M  24.0     NaN     NaN
3             4      Edgar Lindenau Aabye   M  34.0     NaN     NaN
4             5  Christine Jacoba Aaftink   F  21.0   185.0    82.0
…        …                         …  ..   …     …       …
271111   135569                Andrzej ya   M  29.0   179.0    89.0
271112   135570                  Piotr ya   M  27.0   176.0    59.0
271113   135570                  Piotr ya   M  27.0   176.0    59.0
271114   135571        Tomasz Ireneusz ya   M  30.0   185.0    96.0
271115   135571        Tomasz Ireneusz ya   M  34.0   185.0    96.0

                   Team  NOC        Games  Year  Season            City  \
0                 China  CHN  1992 Summer  1992  Summer       Barcelona
1                 China  CHN  2012 Summer  2012  Summer          London
2               Denmark  DEN  1920 Summer  1920  Summer       Antwerpen
3       Denmark/Sweden  DEN  1900 Summer  1900  Summer           Paris
4          Netherlands  NED  1988 Winter  1988  Winter         Calgary
…                    …  …          …    …     …              …
271111        Poland-1  POL  1976 Winter  1976  Winter       Innsbruck
271112          Poland  POL  2014 Winter  2014  Winter           Sochi
271113          Poland  POL  2014 Winter  2014  Winter           Sochi
271114          Poland  POL  1998 Winter  1998  Winter          Nagano
271115          Poland  POL  2002 Winter  2002  Winter  Salt Lake City

                Sport                                      Event Medal
0          Basketball              Basketball Men's Basketball   NaN
1                Judo            Judo Men's Extra-Lightweight   NaN
2            Football                 Football Men's Football   NaN
3          Tug-Of-War            Tug-Of-War Men's Tug-Of-War  Gold
4       Speed Skating       Speed Skating Women's 500 metres   NaN
```

```
…              …                                                …   …
271111          Luge                 Luge Mixed (Men)'s Doubles   NaN
271112    Ski Jumping  Ski Jumping Men's Large Hill, Individual   NaN
271113    Ski Jumping        Ski Jumping Men's Large Hill, Team   NaN
271114      Bobsleigh                       Bobsleigh Men's Four   NaN
271115      Bobsleigh                       Bobsleigh Men's Four   NaN

[271116 rows x 15 columns]
```

0.43 Profilare la memoria in Pandas

[76]:
```
athletes.shape
```

[76]:
```
(271116, 15)
```

[68]:
```
athletes.memory_usage(deep=True)
```

[68]:
```
Index          128
ID         2168928
Name      20697535
Sex       15724728
Age        2168928
Height     2168928
Weight     2168928
Team      17734961
NOC       16266960
Games     18435888
Year       2168928
Season    17080308
City      17563109
Sport     18031019
Event     24146495
Medal      9882241
dtype: int64
```

- le dimensioni sono riportate in bytes

0.44 Profilare la memoria in Pandas

[70]:
```
athletes.memory_usage(deep=True).sum()
```

[70]:
```
186408012
```

- il DataFrame occupa quasi **178 Mb**
- la funzione **memory_usage** può essere utilizzata per calcolare la dimensione di un **DataFrame** o di una **Series**
- **deep=True**
  - riporta una stima precisa, a livello di sistema

– utilizzabile anche con la funzione **info**

0.45 Profilare la memoria in Pandas

[71]:
```
athletes.info(memory_usage='deep')
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 271116 entries, 0 to 271115
Data columns (total 15 columns):
 #   Column  Non-Null Count   Dtype
---  ------  --------------   -----
 0   ID      271116 non-null  int64
 1   Name    271116 non-null  object
 2   Sex     271116 non-null  object
 3   Age     261642 non-null  float64
 4   Height  210945 non-null  float64
 5   Weight  208241 non-null  float64
 6   Team    271116 non-null  object
 7   NOC     271116 non-null  object
 8   Games   271116 non-null  object
 9   Year    271116 non-null  int64
 10  Season  271116 non-null  object
 11  City    271116 non-null  object
 12  Sport   271116 non-null  object
 13  Event   271116 non-null  object
 14  Medal   39783 non-null   object
dtypes: float64(3), int64(2), object(10)
memory usage: 177.8 MB
```

0.46 Ottimizzare la RAM in Pandas: dati categoriali

- **dato categoriale**: i valori assunti sono pochi rispetto all numero di dati totali del **DataFrame**
- primo passo: individuare i dati candidati ad essere trasformati in dati categoriali
- i campi **Medal**, **Season**, **Team** possono essere considerati categoriali?

[72]:
```
athletes['Medal'].unique()
```

[72]:
```
array([nan, 'Gold', 'Bronze', 'Silver'], dtype=object)
```

[73]:
```
athletes['Season'].unique()
```

[73]:
```
array(['Summer', 'Winter'], dtype=object)
```

[74]:
```
athletes['Team'].nunique()
```

[74]:
```
1184
```

0.47 Ottimizzare la RAM in Pandas: dati categoriali

[78]:
```
athletes['Medal'].memory_usage(deep=True)
```

[78]:
```
9882369
```

[79]:
```
athletes['Medal'].astype('category').memory_usage(deep=True)
```

[79]:
```
271539
```

- risparmio ottenuto: **da più di 9 Mb a 0.25 Mb**

0.48 Ottimizzare la RAM in Pandas: valori numerici

[80]:
```
athletes['ID'].min(), athletes['ID'].max()
```

[80]:
```
(1, 135571)
```

- **ID** ha tipo **int64**
- l'intervallo di valori rappresentabili è molto più grande di quello necessario

[81]:
```
import numpy as np
np.iinfo('int64')  # integer info
```

[81]:
```
iinfo(min=-9223372036854775808, max=9223372036854775807, dtype=int64)
```

[82]:
```
athletes['ID'].memory_usage(deep=True)
```

[82]:
```
2169056
```

[83]:
```
athletes['ID'].astype('int32').memory_usage(deep=True)
```

[83]:
```
1084592
```

0.49 Ottimizzare la RAM in Pandas: valori numerici

[84]:
```
athletes['Height'].min(), athletes['Height'].max()
```

[84]:
```
(127.0, 226.0)
```

[85]:
```
athletes['Height'].memory_usage(deep=True)
```

[85]:
```
2169056
```

[86]:
```
athletes['Height'].astype('float16').memory_usage(deep=True)
```

[86]:
```
542360
```

0.50 Ottimizzare la RAM in Pandas: valori numerici

[87]:
```
athletes['Year'].min(), athletes['Year'].max()
```

[87]: (1896, 2016)

[88]:
```
athletes['Year'].memory_usage(deep=True)
```

[88]: 2169056

[89]:
```
athletes['Year'].astype('int16').memory_usage(deep=True)
```

[89]: 542360

[90]:
```
athletes['Year'].astype('category').memory_usage(deep=True)
```

[90]: 272596

- per la categoria **Year** si risparmia di più scegliendo categorie piuttosto che valori numerici
- ovvaimente i dati di tipo categoriale non consentono di effettuare su du essi operazioni aritmetiche

0.51 Ottimizzare la memoria usando "sparse data"

[91]:
```
athletes['Medal'].memory_usage(deep=True)
```

[91]: 9882369

[92]:
```
athletes['Medal'].astype('category').memory_usage(deep=True)
```

[92]: 271539

[93]:
```
athletes['Medal'].astype('Sparse[category]').memory_usage(deep=True)
```

[93]: 199067

0.52 Scegliere i tipi al caricamento dei dati

[94]:
```
schema = {
    'ID': 'int32',
    'Height': 'float16',
    # add all your Column->dtype mappings
    }

athletes = pd.read_csv('DataSets/athlete_events.csv', dtype=schema)
athletes
```

[94]:
```
             ID                      Name Sex   Age  Height  Weight  \
0             1                 A Dijiang   M  24.0   180.0    80.0
1             2                  A Lamusi   M  23.0   170.0    60.0
2             3       Gunnar Nielsen Aaby   M  24.0     NaN     NaN
3             4      Edgar Lindenau Aabye   M  34.0     NaN     NaN
4             5  Christine Jacoba Aaftink   F  21.0   185.0    82.0
…           …                         …  ..   …     …       …
271111   135569                Andrzej ya   M  29.0   179.0    89.0
271112   135570                  Piotr ya   M  27.0   176.0    59.0
271113   135570                  Piotr ya   M  27.0   176.0    59.0
271114   135571        Tomasz Ireneusz ya   M  30.0   185.0    96.0
271115   135571        Tomasz Ireneusz ya   M  34.0   185.0    96.0

                   Team  NOC        Games  Year  Season            City  \
0                 China  CHN  1992 Summer  1992  Summer       Barcelona
1                 China  CHN  2012 Summer  2012  Summer          London
2               Denmark  DEN  1920 Summer  1920  Summer       Antwerpen
3        Denmark/Sweden  DEN  1900 Summer  1900  Summer           Paris
4           Netherlands  NED  1988 Winter  1988  Winter         Calgary
…                     …  …          …   …      …                …
271111         Poland-1  POL  1976 Winter  1976  Winter       Innsbruck
271112           Poland  POL  2014 Winter  2014  Winter           Sochi
271113           Poland  POL  2014 Winter  2014  Winter           Sochi
271114           Poland  POL  1998 Winter  1998  Winter          Nagano
271115           Poland  POL  2002 Winter  2002  Winter  Salt Lake City

                 Sport                                       Event Medal
0           Basketball               Basketball Men's Basketball   NaN
1                Judo             Judo Men's Extra-Lightweight   NaN
2            Football                  Football Men's Football   NaN
3          Tug-Of-War              Tug-Of-War Men's Tug-Of-War  Gold
4        Speed Skating          Speed Skating Women's 500 metres   NaN
…                  …                                         …   …
271111           Luge                Luge Mixed (Men)'s Doubles   NaN
271112    Ski Jumping  Ski Jumping Men's Large Hill, Individual   NaN
271113    Ski Jumping        Ski Jumping Men's Large Hill, Team   NaN
271114      Bobsleigh                      Bobsleigh Men's Four   NaN
271115      Bobsleigh                      Bobsleigh Men's Four   NaN

[271116 rows x 15 columns]
```

0.53 Caricare solo le colonne necessarie

[95]:
```
athletes = pd.read_csv('DataSets/athlete_events.csv', usecols=['NOC', 'Medal'],␣
↪dtype=schema)
athletes.info(memory_usage='deep')
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 271116 entries, 0 to 271115
Data columns (total 2 columns):
 #   Column  Non-Null Count   Dtype 
---  ------  --------------   ----- 
 0   NOC     271116 non-null  object
 1   Medal   39783 non-null   object
dtypes: object(2)
memory usage: 24.9 MB
```

0.54 Caricare un sottoinsieme delle righe

[96]:
```
athletes = pd.read_csv('DataSets/athlete_events.csv', nrows=1000)
len(athletes)
```

[96]: 1000

0.55 Importare i dati in chunks

[97]:
```
# importing the mmodule
import pandas
# Reading the data in chunks
import pandas as pd
chunksize = 100000
for chunk in pd.read_csv('DataSets/athlete_events.csv', chunksize=chunksize):
    # process each chunk here
    print(len(chunk))
```

```
100000
100000
71116
```

[ ]: